Advanced Computer Vision – 4005-753
Home work 2

Karina Damico
Piter Garcia

In this work Harris corner detection was implemented. For better results images were contrast adjusted prior detection, it helped to enhance details on the image that increses the probability of detecting more corners.

```
function [cim, r, c] = harris(im, sigma, thresh, radius)

    dx = [-1 0 1; -1 0 1; -1 0 1];
    dy = dx';
    Ix = conv2(im, dx, 'same');
    Iy = conv2(im, dy, 'same');
    g = fspecial('gaussian',max(1,fix(6*sigma)), sigma);

     Ix2 = conv2(Ix.^2, g, 'same');
     Iy2 = conv2(Iy.^2, g, 'same');
     Ixy = conv2(Ix.*Iy, g, 'same');
     cim = (Ix2.*Iy2 - Ixy.^2)./(Ix2 + Iy2 + eps);
    %must be some +eps (or + something) to avoid division by 0

    sze = 2*radius+1;
    mx = imdilate(cim, strel('square',sze));
    cim = (cim==mx)&(cim>thresh);

    [r,c] = find(cim);
    imagesc(im), hold on, axis off, axis equal, colormap(gray);
    plot(c,r,'r*'),hold off;
end
```

Parameters Definitions

Inputs:
im - image to be processed.
sigma - standard deviation of smoothing Gaussian. Typical values to use might be 1-3.
thresh – threshold.
radius - radius of region considered in non-maximal uppression.

Outputs:
cim - binary image marking corners.
r - row coordinates of corner points.
c - column coordinates of corner points.

For the Harris corner detector, the local structure matrix is smoothed by a Gaussian filter. Harris corner detector is based on the local auto-correlation function of a signal. The local auto-correlation function measures the local changes of the signal with patches shifted by a small amount in different directions.

```
Im1=imadjust(im2double(rgb2gray(imread('PurpleFlowerSmall.jpg'))));
Im2=zeros(200,200);
    for i=20:20:180
     Im2(:,i)=1;
     Im2(:,i+1)=1;
     Im2(i,:)=1;
     Im2(i+1,:)=1;
    end
```

The Harris Corners module identifies corners present within the image using the Harris Corner detection technique. This technique first identifies vertical and horizontal edges using a Sobel type edge detector. Those edges are then blurred to reduce the effect of any image noise. The resulting edges are then combined together to form an energy map that contains peaks and valleys. The peaks indicate the presence of a corner.

```
Im3=imadjust(im2double(rgb2gray(imread('kitty.jpg'))));
Im4=imadjust(im2double(rgb2gray(imread('theThing.jpg'))));

harris(Im1,1,0.1,1);
harris(Im2,1,0.5,1);
harris(Im3,1,0.05,1);
harris(Im4,2,0.04,2);
```

Results directly depend on input parameters of the Harris function.

Threshold - lower values allow more subtle corners to appear whereas higher values reduce the number of corners identified.

Data Influence - The algorithm to consider color or not, color can help identify corners between colors but adds additional processing. While the traditional Harris corner detector can work well, this also can be slow for some applications. If speed is a concern of accuracy, the Fast Harris enables the fast Harris corner detector which optimizes the Harris detector by approximating some steps to achieve additional speed.

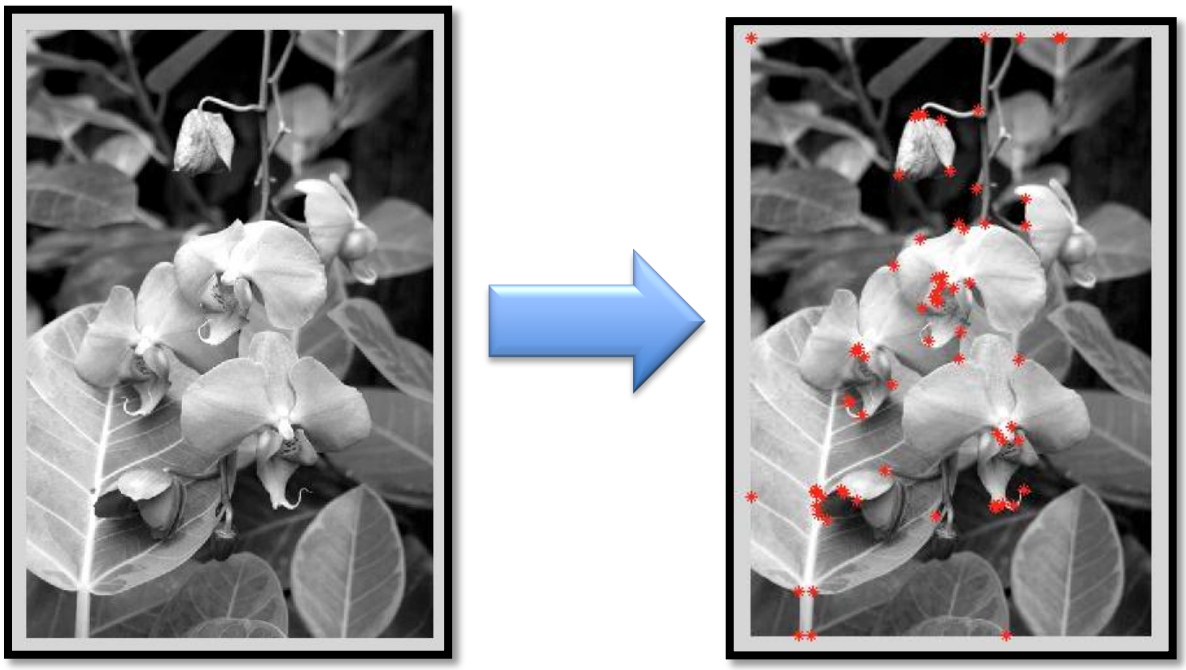
Color - The color to display the detected corner points on top of the current image.
Shape - The shape to use when displaying the detected corner points.

Corner Isolation - eliminates points close to each other select an appropriate corner isolation number. This will ensure that points are at least X pixels from each other and prevents tight clustering of detected points.

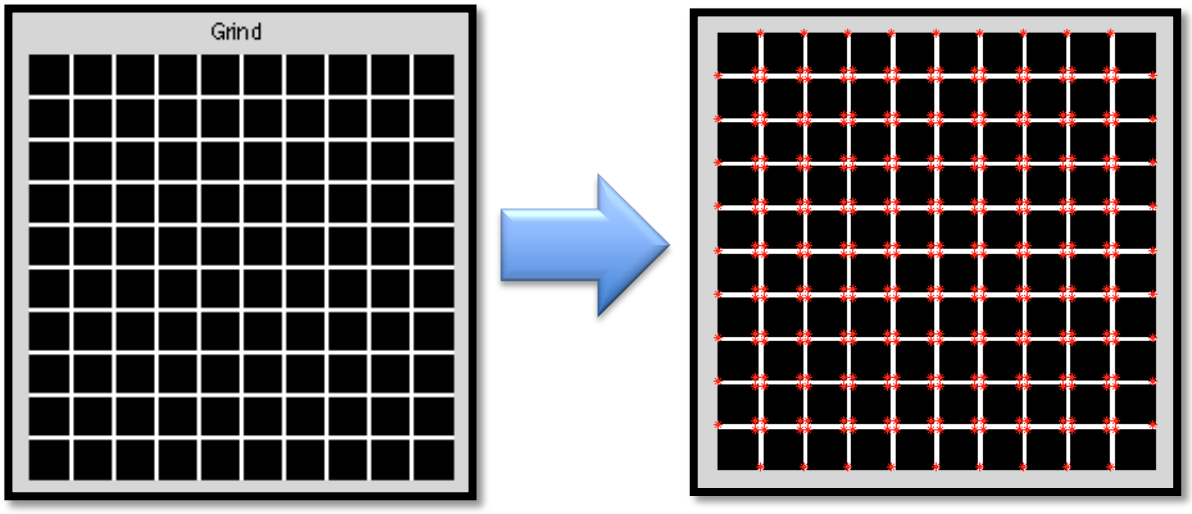
Overall varying the sigma, threshold and radius values we get the results explained above (see second example).

The followng results were obtained:

1) `harris(Im1,1,0.1,1);` → `sigma=1; threshold=0.1; radius=1;`

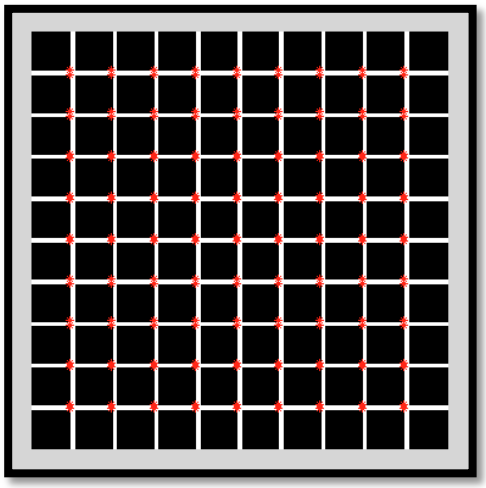


2) `harris(Im2,1,0.5,1);` → `sigma=1; threshold=0.5; radius=1;`



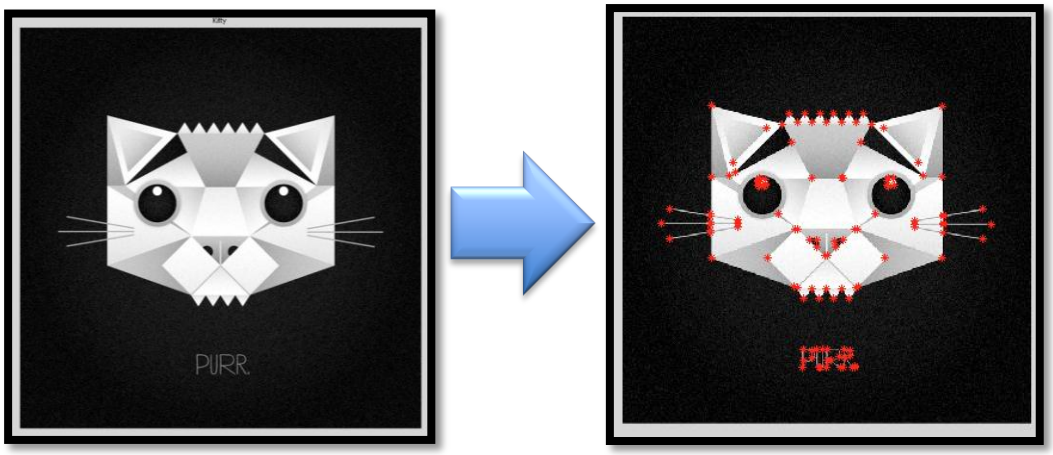
Finding corners on the grind is the best demonstration. However, even on such straight and forward example not all the corners can be detected if chosen parameters are not adjusted.

Following example demonstrates the result of Harris corner detection for sigma=2 and radius=2. Threshold will not affect this image as much as the strictly black and white image does, but in general case (e.g. the flower example) reducing threshold increases the amount of detected corners that can lead to detecting false corners. However, increasing the threshold decreases the amount of detected corners by making them not strong enough and provoking them

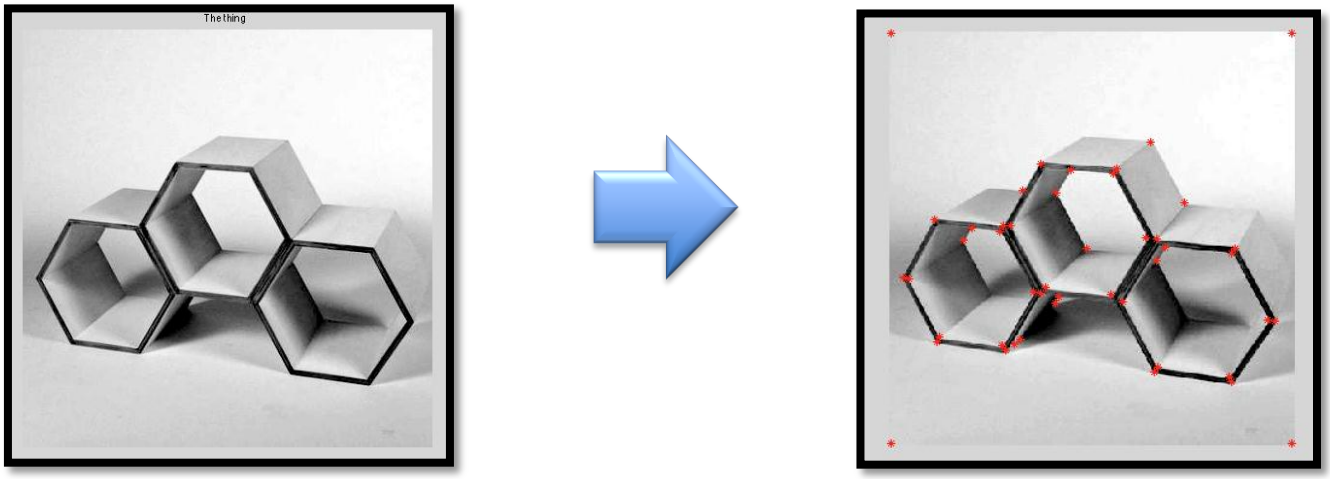


to be missed:

3) `harris(Im3,1,0.05,1);` → `sigma=1; threshold=0.05; radius=1;`



4) `harris(Im4,2,0.04,2);` → `sigma=2; threshold=0.04; radius=2;`



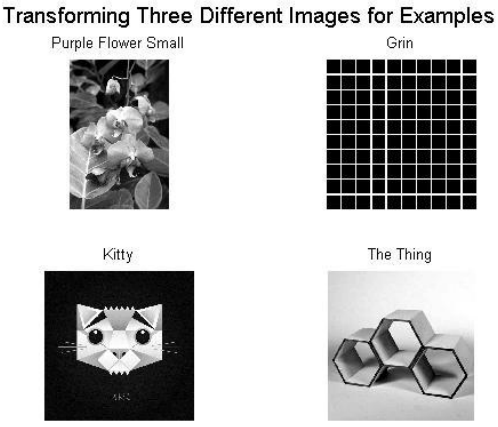
The quality of the image is not good, and to avoid detecting false corners the sigma and radius were increased (sigma to increase the smoothness of the image).

To prevent too many corner features lumping together closely, a non-maximal suppression process on the corner response image was carried out to suppress weak corners around the stronger ones. This is then followed by a thresholding process. Altogether, the Harris

corner detector requires three additional parameters such as the constant, the radius, of the neighborhood region for suppressing weak corners, and the threshold value.

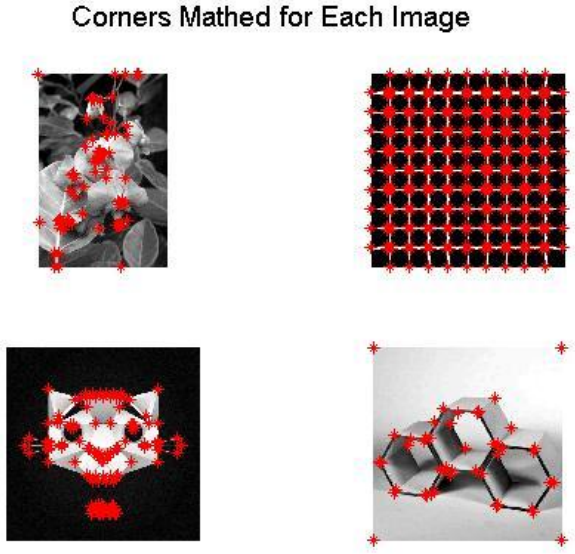
```
figure,
subplot(1,3,1),imshow(imread('PurpleFlow
erSmall.jpg')), title('Purple Flower
Small');
subplot(1,3,2),imshow(imread('Kitty.jpg'
)), title('Kitty');
subplot(1,3,3),imshow(imread('theThing.j
pg')), title('The Thing');
suptitle('Loading Three Different Images
for Examples');
```


```
figure,
subplot(2,2,1),imshow(Im1),
title('Purple Flower Small');
subplot(2,2,2),imshow(Im2),
title('Grin');
subplot(2,2,3),imshow(Im3), title('Kitty');
subplot(2,2,4),imshow(Im4), title('The Thing');
suptitle('Transforming Three Different Images for Examples');
```

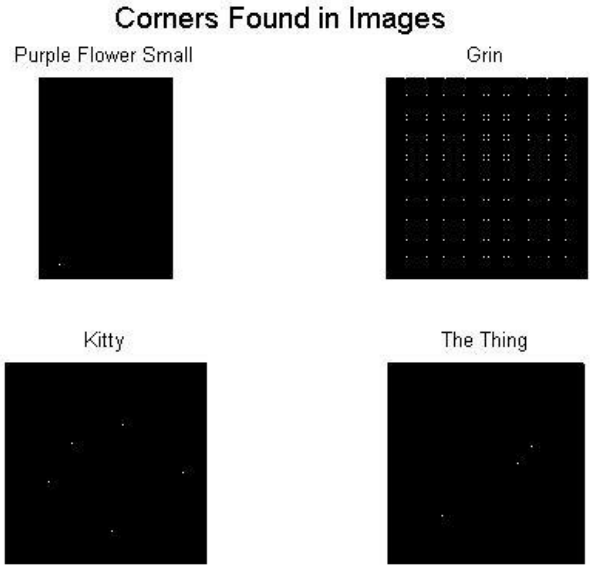



```
figure,
subplot(2,2,1),imshow(harris(Im1,1,0.1,1)
), title('Purple Flower Small');
subplot(2,2,2),imshow(harris(Im2,1,0.5,1)
), title('Grin');
subplot(2,2,3),imshow(harris(Im3,1,0.05,1
)), title('Kitty');
subplot(2,2,4),imshow(harris(Im4,2,0.04,2
)), title('The Thing');
suptitle('Corners Found in Images');
```

```
figure,
subplot(2,2,1),title('Purple Flower
Small');
harris(Im1,1,0.1,1);
subplot(2,2,2),title('Kitty');
harris(Im2,1,0.5,1);
subplot(2,2,3),title('Kitty');
harris(Im3,1,0.05,1);
subplot(2,2,4),title('The Thing');
harris(Im4,2,0.04,2);
suptitle('Corners Mathed for Each Image');
```



Our Harris Corner Detector behaves as a mathematical operator to match features in an image. These features are simple to compute by the rotation, scale and illumination variation independent of our algorithm. Our aim in this assignment is to find little patches of image that generate a large variation when moved around.



As observed on the following examples in this assignment, the Grin image to the left is the window we have chosen. Moving it around would not show much of variation. This is because the difference between the window and the original image below it is very low. Therefore, we cannot really tell if the window belongs to any position. However, when we move the window too much we are bound to see a big difference. Even the little movement of the window produces a noticeable difference.

If thresh and radius were omitted from the argument list only the cim parameter would return as a raw corner strength image. Then, looking at the values within the parameter cim is possible to determine the appropriate threshold value to be used. In fact, the Harris corner strength varies with the intensity gradient raised to the 4th power. Small changes in input image contrast result in huge changes in the appropriate threshold as shown in the preivous pictures examples in this assignment.

Concluding, the Harris Corner Detector is the right mathematical way of determining the right windows that produce large variations when these are moved in any direction. Each window has an associated score, and it is based on this score that we are able to figure out which ones are corners and which ones are not.